

Frames in Playwright

Target Locators

PLAYWRIGHT · FRAMES

Understand how to identify frames, interact with elements inside them, work with nested frames, and return to the main page.

WHAT IS A FRAME?

A frame is another DOM embedded inside the main page DOM.

WHY DO WE NEED TO HANDLE IT?

To interact with elements that belong to the frame.

HOW DOES PLAYWRIGHT HANDLE IT?

Playwright does not require a Selenium-style driver focus switch. Use `frameLocator()` or a `Frame` object to work with the frame content.

The Golden Circle

What • Why • How

WHAT

A frame is an embedded browsing context containing its own DOM.

WHY

Elements inside a frame must be located within that frame context.

HOW

Use `frameLocator()` for locator-based automation or `page.frame()` when a `Frame` object is needed.

How to Identify an iframe

Inspect the page structure

Inspect the DOM

Look for an `<iframe>` element in the page structure.

Browser DevTools

Use the Elements panel to identify the iframe and inspect attributes such as id, name and src.

Playwright

A frame can then be targeted through an iframe locator or identified through the page's Frame objects.

Structure of a Frame

HTML inside another HTML

The main page contains an iframe element.

The iframe loads another document with its own DOM.

Elements inside the iframe are accessed through the frame context.

How to Handle It?

Playwright approach

Recommended approach

Use `frameLocator()` when your goal is to interact with elements inside an `iframe`.

Basic syntax TypeScript

```
const frame = page.frameLocator('iframe');  
await frame.getByRole('button', { name: 'Click Me' }).click();
```

Key difference

There is no `driver.switchTo().frame()` step in Playwright. The locator is scoped directly to the frame.

Ways to Access a Frame

Playwright patterns

```
const frame = page.frameLocator('iframe');
```

```
const frame = page.frameLocator('#frame1');
```

```
const frame = page.frame({ name: 'frameName' });
```

```
const allFrames = page.frames();
```

Frame Locator

Interact directly inside the iframe

Locate the iframe

Create a `frameLocator` using the `iframe` selector.

Locate the element

Use normal Playwright locators from the `frameLocator`.

Example TypeScript

```
const frame = page.frameLocator('iframe');
await frame.getByText('Click Me').click();
const text = await frame.locator('body').innerText();
```

Best practice

Prefer `frameLocator()` for normal UI automation because it keeps the frame boundary inside the locator.

Frame Object

When you need the Frame API

Get a Frame object

Use `page.frame()` with a frame name or URL when that information is available.

By name TypeScript

```
const frame = page.frame({ name: 'frameName' });  
await frame?.getByRole('button').click();
```

By URL TypeScript

```
const frame = page.frame({ url: '/frame\.html/' });  
await frame?.locator('button').click();
```

Important

`page.frame()` can return null when the requested frame is not found, so handle the value safely.

Nested Frames

Frame inside another frame

Parent frame

A nested frame is an iframe inside another iframe.

Playwright approach

Chain `frameLocator()` calls to move through the frame hierarchy.

Nested frame TypeScript

```
const child = page
  .frameLocator('#parentFrame')
  .frameLocator('#childFrame');

await child.getByRole('button', { name: 'Click Me' }).click();
```

Returning to Main Content

No defaultContent() required

Selenium

Selenium uses `driver.switchTo().defaultContent()` to return to the main page.

Playwright

With `frameLocator()`, simply use a page locator again for elements in the main document.

Example TypeScript

```
const frame = page.frameLocator('#frame1');
await frame.getByRole('button').click();

// Back to main page
await page.getByRole('button', { name: 'Main Button' }).click();
```

Key idea

Playwright scopes the locator instead of changing a global driver focus.

Selenium → Playwright

Concept mapping

Switch to frame

Frame by id/name

Frame by WebElement

Nested frame

Return to main

Classroom Activity

Frame + Alert with Playwright

Launch

https://www.w3schools.com/js/tryit.asp?filename=tryjs_confirm

Action

The Try It example is displayed inside the page's iframe.

Handle the frame

Use `frameLocator()` to locate the Try It button inside the iframe.

Handle the alert

Register `page.once('dialog', ...)` before clicking the button, then accept or dismiss the confirmation.

Playwright pattern TypeScript

```
page.once('dialog', async dialog => {
  console.log(dialog.message());
  await dialog.accept(); // or dialog.dismiss()
});

const frame = page.frameLocator('iframe');
await frame.getByRole('button', { name: 'Try it' }).click();
```

Verification

After the dialog action, verify the resulting text or page state using a normal Playwright locator.

Summary

Frames in Playwright

Iframe → inline frame → DOM inside another DOM.

Use `frameLocator()` for locator-based interaction inside an iframe.

Use `page.frame()` when a `Frame` object is required.

`page.frames()` returns the frames available on the page.

Nested frames can be handled by chaining `frameLocator()`.

No driver focus switch or `defaultContent()` is required with `frameLocator()`.